single server used in postman

Multiple server used in Eureka server

Phase1:

>Java Basics(OOP, Collections)

>JavaFX controls & events

Phase2:

>Servelt lifecyle

>Request & Response

>JSP tags& Expression Language

Phase3:

>JDBC(Database)

>MVC Architecture

>Mini Project: Student Management

Technology	What it is	Where it runs	Used for
JavaFX	Desktop UI Framework	Your computer	Desktop apps(calculator, student app)
Servlets	Java backend logic	Server(Tomcat)	Handle requests, business logic

JSP HTML+Java Server(Tomcat) Dynamic web pages

HTML/CSS/JS Frontend Browser UI for web pages

>JavaFX->Desktop apps

>JSP+Servlets-> Web app (runs on browser via server)

Software you need (Install)

1. JDK8 or above-JDK 17
2. Eclipse IDE(Enterprise Edition)
3. Apache Tomcat Server(v9 recommended)
4. Web Browser(Chrome/Edge)

Install & Setup

step1: Install JDK

Download: https://www.oracle.com/java/technologies/downloads

>Install-> note the path

>Verify:cmd-> java -version

Step2:Install Eclipse(Important version)

Download Eclipse IDE for Enterprise Java and Web Developers.

https://www.eclipse.org/downloads

Why this version?

it supports Servlets & JSP

Has Dynamic Web Project

step3: Install Tomcat Server

Download:https://tomct.apache.org/download-90.cgi

Extract Zip

Remember the folder location

Part1: JavaFX(Desktop Application)

What are JavaFX Controls?

Controls are UI components that allow the user to view data or interact with the application.

Ex:

>Button-> click actions

>TextField->enter text

>Lable->display text

>CheckBox->selection option

>RadioButton->Choose one option

>ComboBox-> dropdown selection

All controls come from:

javafx.scene.control

Commonly used JavaFx Controls:

Label: Display text(cannot edit)

ex:

```
Label label=new Label("Welcome to JavaFX");
```

2. Button: Performs an action when clicked

ex:

```
Button btn=new Button("Click Me");
```

3. TextField: Single-line user input

```
TextField tf=new TextField();
tf.setPromptText("Enter your name");
```

4. PasswordField: Hides user input

```
PasswordField pf=new PasswordField();
```

5. TextArea: Multi-line input

ex:

```
TextArea ta=new TextArea();
```

6. Checkbox: Multiple selections allowed.

```
Checkbox cb=new Checkbox("Java");
```

7.RadioButton: Only one option can be selected(Group required)

ex:

```
ToggleGroup group=new ToggleGroup();
RadioButton r1=new RadioButton("Male");
RadioButton r2=new RadioButton("Female");
r1.setToggleGroup(group);
r2.setToggleGroup(group);
```

8. ComboBox(Drodown)

ex:

```
ComboBox<String> combo=new ComboBox<>();
```

```
combo.getItems().addAll("CSE", "ECE", "IT");
```

JavaFX Layouts: Controls must be placed inside Layouts

Layout	Use
VBox	Vertical alignment
HBox	Horizontal alignment
GridPane	Row/Column(Forms)
BorderPane	Top, Bottom, Center

ex:

```
VBox root=new VBox(10); //spacing=10
root.getChildren().addAll(label, btn);
```

Visual idea:

```
Label
(<-------10px gap----->
Button
```

```
Stage(window)
|-Scene
|-|-VBox(root)
| | |-Label
| | |-Button
```

4. JavaFX Events:

What is an Event?

An event occurs when the user interacts with a control

Examples:

Button click

Mouse movement

Key press

5. Button click Event: Using lambda expression

ex:

```
btn.setOnAction(e->{
   System.out.println("Button Clicked!");
});
```

Real Example(Button + TextField)

```
Button btn=new Button("Submit");
TextField tf=new TextField();
btn.setOnAction(e->{
  String name=tf.getText();
  System.out.println("Hello "+name);
});
```

6. Mouse Events:

Event	Method
Mouse Click	setOnMouseClicked
Mouse Enter	setOnMouseEntered
Mouse Exit	setOnMouseExited

ex:

```
btn.setOnMouseEntered(e->btn.setText("Mouse Inside"));
btn.setOnMouseExited(e->btn.setText("Click Me"));
```

7. Keyboard Events:

```
tf.setOnKeyReleased(e->{
   System.out.println("Key Released");
});
```

8. ComboBox Selection Event:

```
combo.setOnAction(e->{
  String vlaue=combo.getValue();
  System.out.println("Selected: " + value);
});
```

9. Checkbox Event

```
cb.setOnAction(e->{
  if(cb.isSelected())
    System.out.println("Chceked");
  else
     System.out.println("Unchecked");
});
```

Simple JavaFx Example(Your FirstApp)

```
import javafx.application.Application;  //Base class for any JavaFxapp.
import javafx.scene.Scene; //canvas for UI
```

```
import javafx.scene.control.Label; //Ui element that shows text.
import javafx.stage.Stage; //Represents the main window on your desktop
public class MainApp extends Application{
  @Override
  public void start(Stage stage){
    Label label=new Label("Hello JavaFx!");
    Scene scene=new Scene(label, 300,200);
    stage.setTitle("My First JAvaFx App");
    stage.setScene(scene);
    stage.show();
   }
public static void main(String[] args){
   launch(args);
  }
}
```

JavaFxConcept	Analogy
Stage	Window on your desktop
Scene	Paper inside window
Label	Text or sign on a paper
start()	Desinger putting things on paper
main()+launch()	Factory starting the designer & window

Step1:

```
import javafx.application.Application;
import javafx.scene.Scene;
```

```
import javafx.scene.control.Label;
import javafx.stage.Stage;
```

Application: Base class for any JAvaFX application

Stage: Represents the main window

Scene: Holds all UI Components

Label: Displays text on the screen

Step2: Class Declaration

```
public class MainApp extends Application{
```

MainApp is your main class

It extends Application, which is mandatory for JavaFx apps.

JavaFX automatically calls certain methods from this class

Step3: start() Method(Main JavaFX method)

```
@Override
public void start(Stag stage){
```

This method is automatically called when the app starts

Stage stage-> The main window of the application

Step4:

```
Label label=new Label("Hello JavaFx!");
```

Create a Label control

Display the text "Hello JavaFX!" on the screen

This is your UI content.

Step5:

Scene scene=new Scene(label, 300,200);

A Scene is like a container for UI elements

Parameters:

label: root UI element

300: width(pixel)

200: height(pixels)

Stage->Scene->UI Controls(Label)

Step6:stage.setTitle("My First JAvaFx App");

Sets the title of the window(as shown at the top)

Step7: stage.setScene(scene);

Attachign Scene to Stage

Connects the scene (UI) to the stage(window)

Step8: Displaying the window

stage.show()

Web Develpment using Servelts & JSP:

How Web Java Wroks:

Browser->JSP->Servelt->Database

Browser<-HTML<-JSP<-Servelt

When you open a website:

Browser(Chrome) sends a request

Server processes it

Server sends back a response(HTML page)

JSP and Servlet's live on the SERVER side

What is a Servlet?

It is a Java Program that runs on a server.

Think of a Servlet like a smart office clerk:

>You given a request(form, login, data)

>Clerk process it using logic

>Clerk sends back a result

What does a Servlet do?

>Receives data from browser(form)

>Performs logic(Validation, calculations)

>Connects to database

>Sends response(HTML)

Simple Servlet Example:

```
public class HelloServlet extends HttpServlet{
  protected void doGet(HttpServeltRequest request, HttpServletResponse
response){
      response.getWriter().println("Hello Student!");
 }
```

}

What is JSP?

JSP=HTML+Java

JSP is a smart HTML Page.

>Looks like HTML

>Can write small java code inside it.

Real life example:

A printed form where some fields are autofilled->Name, data, marks etc.

The auto-fill logic-JSP

Example:

```
<html>
<body>
  <h2>Welcome Student</h2>
  Today's DATE: <% new java.util.Date()%>
</body>
</html>
```

<%=%>: prints Java output on page.

HTML+Java together

Difference between Servlet and JSP:

Feature	Servlet	JSP
Type	Java class	HTML page

Used for	Logic(processing)	Display(UI)
Writing style	Pure Java	HTML+Java
Best for	Backend work	Frontend display

Web Development Using Servlets & JSP:

Create Web Project:

1. New file-> Dynamic Web Project
2. Project Name: StudentWebApp
3. Target Runtime->Apache Tomcat
4. Finish

Configure Tomcat in Eclipse:

1. Window->Show View->Servers
2. Click No servers available
3. Select Apache Tomcat v9
4. Browse Tomcat folder

5.Finish

First Servlet Example:

Create Servlet:

1. Right click project
2. New->Servlet
3. Package: com.student
4. ServletName: HelloServlet

5.Finsih

ServletCode:

```
import java.io.IOException;  //Used to handle input/output error

import java.servlet.*;

import javax.servlet.http.*; // These are Servlet API libraries. They give us HttpSevlet, HttpServletRequest, HttpServletResponse. Without these imports-> servlet will not work


public class HelloServlet extends HttpServlet{

  protected void doGet(HttpServeltRequest request, HttpServletResponse response)

        throws ServletException, IOException{

            response.setContentType("text/html");

            response.getWriter().println("<h1>Hello from Servlet</h1>);

  }

}
```

Example 2 :

```
package com.student;


import java.io.IOException;

import java.io.PrintWriter;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request, HttpServletResponse
response)
          throws ServletException, IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();
      out.println("<h1>Hello from Servlet</h1>");
   }
}
```

Getting error in Server :

open cmd type : netstat -ano | findstr 8080

- check the 8080 is Listening
- Change server

Go to server

-> double on Apache tomcat server

-> Replace the 8080 to 8081 to save

-> Run the program

output shows on web : Hello from Servlet

Mini Project on JSP+ Servlets

application : webapp

structure :

```
Webapp
 ├── Java Resources
 |    └── src/main/java
 |         └── com.student
 |              └── StudentServlet.java
 ├── src
 |    └── main
 |         └── webapp
 |              ├── index.jsp
 |              ├── result.jsp
 |              └── WEB-INF
```

index.jsp code:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head>
   <title>Student Form</title>
```

```
</head>
<body>

<h2>Student Registration</h2>

<form action="student" method="post">
   Name: <input type="text" name="name" /><br><br>
   Age: <input type="text" name="age" /><br><br>
   <input type="submit" value="Submit" />
</form>

</body>
</html>
```

result.jsp code :

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head>
   <title>Result</title>
</head>
<body>

<h2>Student Details</h2>
```

```
Name: ${studentName} <br>
Age: ${studentAge}

</body>
</html>
```

StudentServlet.java code

```
package com.student;

import java.io.IOException;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/student")
public class StudentServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request, HttpServletResponse
response)
          throws ServletException, IOException {

      String name = request.getParameter("name");
      String age = request.getParameter("age");

      request.setAttribute("studentName", name);
```

```
        request.setAttribute("studentAge", age);

        RequestDispatcher rd = request.getRequestDispatcher("result.jsp");
        rd.forward(request, response);
    }
}
```

Run the code using Server to choose tomcat server